INTRODUCTION TO KUBERNETES AND ORCHESTRATION

What is Kubernetes

1. Kubernetes is a free and open-source system for autoscaling, autohealing, deployment & load balancing.
2. It's a tool for managing the containers
3. It is also known as orchestration tool
4. Orchestration ---> container management tool

Orchestration simply means clustering any number of containers which are running on different network

1. Docker swarm ---> specific to managed docker containers
2. Kubernetes ------> managed any container

History

1. google developed an internal system called borg (later named as omega) to deploy and manage thousands of google applications and services on their cluster.
2. In 2014 google introduced Kubernetes an open-source platform written in golang and later donated to CNCF (cloud native computing foundation)

Online Platform for K8s

1. Kubernetes playground (killerkoda)
2. play with k8s
3. play with Kubernetes classroom

Cloud Based K8s Services

1. GKE (google Kubernetes services)
2. AKS (azure Kubernetes services)
3. EKS (elastic Kubernetes services)

Kubernetes Installation Tools

1. Minikube
2. kubeadm
3. kind

Problems With Scaling Up the Containers

1. containers cannot communicate with each other
2. Autoscaling and load balancing was not possible
3. containers had to be managed carefully

Feature of Kubernetes

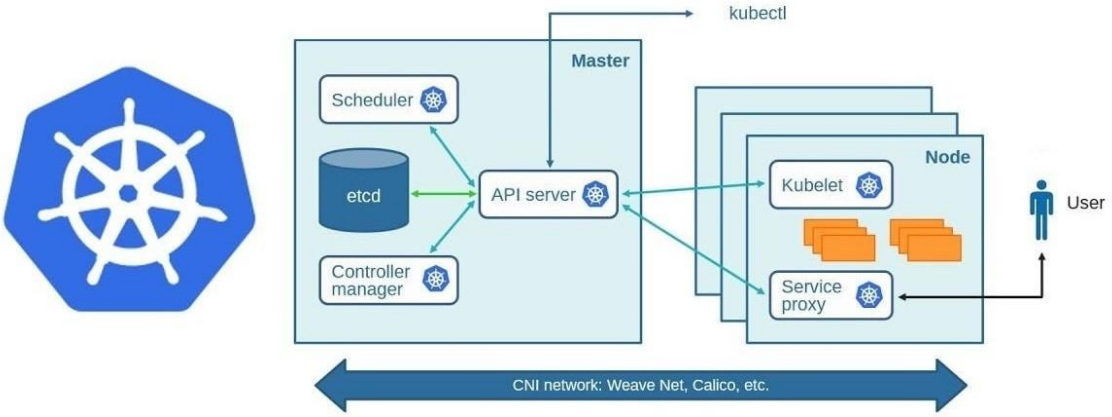
1. Orchestration
2. Autoscaling (vertical / horizontal)
3. Auto-healing
4. load balancing
5. platform independent
6. fault tolerance
7. Rollback
8. Health monitoring of containers
9. Batch execution

Difference between the kubernetes & Docker swarm

Feature	Kubernetes	Docker Swarm
Difficulty	Harder to learn	Easy to learn
Setup	Complex	Simple
Scaling	Very powerful	Basic scaling
Networking	Advanced	Simple
Auto-healing	Strong	Limited
Load Balancing	Advanced	Basic
Best For	Large production apps	Small/simple projects
Community Support	Huge	Smaller
Cloud Support	Supported everywhere	Limited support
Speed	Slightly slower setup	Faster setup

KUBERNETES ARCHITECTURE

Cluster ----> Node -----> Pod -----> container ----> microservice (application)



CONTROL-PLANE COMPONENTS (MASTER-NODE)

1.API Server

1. API Server is the main entry point of Kubernetes Control Plane. Users, kubectl, and other components communicate through it.
2. When we apply a YAML/JSON file using kubectl apply, the request first goes to the API Server.
3. API Server validates the request and stores cluster information in the ETCD database.
4. It acts like the front-end and communication hub between all Kubernetes components.

2.ETCD Server

1. ETCD works as the database of Kubernetes cluster.
2. It stores all cluster information in key-value format, such as Pods, Nodes, ConfigMaps, Secrets, and cluster status.
3. API Server reads and writes all data to ETCD, making it the source of truth for the cluster.
4. ETCD is highly available, secure, and fully replicated so cluster data remains safe and consistent.

3.Kube Scheduler

1. Scheduler watches for newly created Pods that do not have any node assigned.
2. After detecting a new Pod, it checks available Worker Nodes.
3. It selects the best suitable node based on CPU, memory, resources, and policies.
4. Once selected, Scheduler informs the API Server about the assigned node for that Pod.

4.Controller Manager

1. Controller Manager continuously checks whether the actual state matches the desired state.
2. If something goes wrong (Pod crash, Node failure), it automatically tries to fix it.
3. It contains multiple controllers like:
    - Node Controller
    - Replication Controller
    - Endpoint Controller
4. In cloud environments it uses Cloud Controller Manager, and in non-cloud environments it uses Kube Controller Manager.

WORKER-NODE COMPONENTS

1. Kubelet

1. Kubelet is an agent running on every Worker Node in Kubernetes.
2. It continuously listens to the API Server for Pod creation and management instructions.
3. Kubelet creates and manages Pods and containers on the Worker Node.
4. It sends the Pod status and health reports (success/failure) back to the Master Node.

2. Kube Proxy

1. Kube Proxy is responsible for network communication inside the cluster.
2. It helps Pods communicate with other Pods and Services using networking rules.
3. It runs on every Worker Node and manages traffic forwarding.
4. It maintains network rules so each Pod can be accessed properly using its IP address.

3. Container Engine

1. Container Engine is responsible for creating and running containers inside Pods.
2. It works together with Kubelet to manage container operations.
3. It pulls container images from registries like Docker Hub.
4. It starts, stops, and exposes containers on the ports defined in the manifest file.

4. Pods

1. Pod is the smallest deployment unit in Kubernetes.
2. A Pod can contain one or more tightly coupled containers running together.
3. Kubernetes manages Pods, not individual containers directly.
4. Usually, one Pod contains one container for better fault isolation and management.

Multi-Container Pods

1. Multiple containers inside the same Pod can share the same network and memory space.
2. Containers inside a Pod can communicate with each other using localhost.
3. All containers in the Pod can share the same storage volume/data.
4. Containers inside a Pod are deployed and managed in an all-or-nothing manner (if Pod fails, all containers are affected).
5. Entire Pod runs on the same Worker Node, and the Scheduler decides on which node the Pod will run.

WORKING FLOW OF Kubectl COMMAND

1. kubectl is a command-line tool used by users/admins to interact with the Kubernetes cluster.
2. When we run commands like:
   kubectl apply -f pod.yaml ------> the request first goes to the API Server.
3. kubectl does not directly create Pods or containers.
   It only sends instructions/requests to the Control Plane through the API Server.
4. API Server processes the request and coordinates with:
   - ETCD → stores cluster data
   - Scheduler → selects node
   - Controller Manager → maintains desired state
   - Kubelet → creates Pod on Worker Node

WORKING FLOW OF KUBERNETES

1. User applies YAML file using kubectl apply -f pod.yaml.
2. Request goes to the API Server.
3. API Server validates the request and stores cluster data in ETCD.
4. Scheduler detects a new Pod without a node assignment.
5. Scheduler selects the best Worker Node and updates the API Server.
6. Controller Manager ensures the desired Pod is running properly.
7. Kubelet on Worker Node receives instructions from API Server.
8. Container Runtime (Docker/containerd) creates and runs the container.
9. Pod becomes Running and status is updated back into ETCD through API Server.

PODS LIFECYCLE

1. Pending

1. Pod is created but not yet running.
2. Scheduler is finding the best Worker Node.
3. Images may still be downloading.
4. Pod is waiting for required resources.

2. Running

1. Pod is successfully assigned to a Node.
2. Containers inside the Pod are created and started.
3. Application is now running properly.
4. Kubelet continuously monitors the Pod.

3. Succeeded

1. All containers completed their work successfully.
2. Pod stopped without any error.
3. Mostly used in batch jobs or one-time tasks.
4. Pod will not restart again automatically.

4. Failed

1. One or more containers stopped with an error.
2. Application inside Pod crashed or failed.
3. Kubernetes may create a new Pod depending on restart policy.
4. Failed Pod status is stored in ETCD.

5. Unknown

1. Kubernetes cannot determine Pod status.
2. Usually happens due to Node or network issues.
3. API Server cannot communicate with Kubelet.
4. Pod state becomes temporarily unknown.
